Profile-Based Matching Algorithm

Project Overview:-

The **Intelligent Hybrid Recommendation System** aims to transform how individuals discover professional and personal connections by leveraging advanced **Natural Language Processing (NLP)** and adaptive **Machine Learning**. This unified system integrates semantic text analysis, personality profiling (MBTI), and real-time behavioral feedback to create a robust and highly personalized matching experience.

The project focuses on developing an **intelligent compatibility ecosystem** that interprets the nuance of human stories—analyzing unstructured bios, work styles, and personal values rather than relying solely on rigid, static filters. By prioritizing **semantic resonance** and **complementary traits** over simple keyword matching, the system fosters authentic and meaningful interactions.

The goal of this project is to design and execute a **self-learning recommendation framework** that understands complex user profiles, adapts to individual preferences through continuous feedback loops, and enhances the quality of matches to build lasting professional and personal relationships.

Problem Statement:-

Develop a Python-based matching algorithm that calculates compatibility scores between users based on a holistic profile (combining Professional Goals and Personal Traits). The system must:

1. Parse Unstructured Data: Use NLP to quantify similarity between free-text fields (e.g., *"A little about me..."*).
2. Implement Hybrid Logic: Combine text similarity, MBTI logic, and demographic weights into a single score.
3. Learn from Feedback: Implement a mechanism where user actions (Accept/Reject) update the weighting algorithm to improve future recommendations.

Technical Modules:-

1: NLP & Semantic Analysis (The "Understanding" Layer)

- **Objective:** Enable the machine to "read" and compare user bios and professional summaries.

© 2025 UnloxAcademy. All Rights Reserved.
Made by recycled paper, because we care.

- **Implementation:**
    - Preprocessing**:** Clean raw text data by removing stopwords, punctuation, and performing lemmatization on fields like *"About Me"* and *"Professional Summary."*
    - Vectorization**:** Convert the cleaned text into numerical vector embeddings using TF-IDF (for basic implementation) or BERT/Sentence Transformers (for advanced implementation).
    - Similarity Calculation**:** Use Cosine Similarity to mathematically measure the distance (similarity) between User A's text vector and User B's text vector.

2: Profile Scoring Engine (The "Logic" Layer)

- **Objective:** Synthesize multiple data points into a single Compatibility Score (0-100%).
- **Implementation:**
    - MBTI Logic: Define a static rule dictionary or matrix that scores personality pairings (e.g., if UserA='INTJ' and UserB='ENFP': score = 100).
    - Weighted Aggregation: Combine the Text Similarity score, MBTI score, and Location data using a weighted formula.
    - Core Formula:
      $$TotalScore = (w_1 \times TextSim) + (w_2 \times MBTIMatch) + (w_3 \times Location)$$

3: Adaptive Feedback Loop (The "ML" Layer)

- **Objective:** Create a system that learns from user behavior to improve recommendation accuracy over time.
- **Implementation:**
    - Data Collection: Capture user interactions where they rate a match (e.g., *Thumbs Up* = 1, *Thumbs Down* = 0).
    - Dynamic Weight Adjustment: Train a Regression Model (or implement a Gradient Descent logic) to adjust the weights (w_1, w_2) dynamically for specific users.
    - Goal: If a user consistently rejects matches with high "MBTI scores" but accepts matches with high "Professional Similarity," the model must automatically lower the MBTI weight (w_2) for that specific user.

© 2025 UnloxAcademy. All Rights Reserved.
Made by recycled paper, because we care.

Tools Expected

Category	Recommended Tools & Libraries
Programming Language	Python
NLP Libraries	spaCy, NLTK, or HuggingFace Transformers
Machine Learning	Scikit-learn
Data Manipulation	Pandas
Visualization	Matplotlib or Seaborn

Expected Deliverables

1. **Data Pipeline Script:** Code that generates or cleans the user profile data.
2. **Matching Algorithm:** A Python function that takes two User IDs and outputs a **Compatibility Score (0-100%)**.
3. **Performance Analysis:** A report showing how the "Feedback Loop" improved accuracy over time (e.g., "Initially 40% acceptance rate, improved to 65% after training").
4. **UI Demo:** A simple interface (replicating the screenshots) where a user enters data and sees a list of top 5 matches.

5. Dataset: with the dataset the zip called (num)-indian-car-market-capstone-2025 making the data sample training

© 2025 UnloxAcademy. All Rights Reserved.
Made by recycled paper, because we care.